Assignment 8

Scalability and Reliability Improvements in a GUI Chat Application

Name: Madhab Boro

Roll No.: CSB24055

Assignment: 8

---

1. Objective

The objective of this assignment is to improve the scalability and reliability of the GUI-based chat application developed in Assignment 7, without changing the existing TCP communication protocol. The application was evaluated under different client loads, and performance metrics were collected to analyze its behaviour.

2. Scalability Improvements

The following scalability improvements were implemented on top of the Assignment 7 codebase:

- Configuration values (host, port, timeouts, message limits, etc.) moved out of the source code into an external config.json file.
- The application was successfully tested with 5, 8 and 10 concurrent clients.
- Client handling was improved to better support multiple simultaneous connections.
- Performance statistics are now generated automatically for every test run.

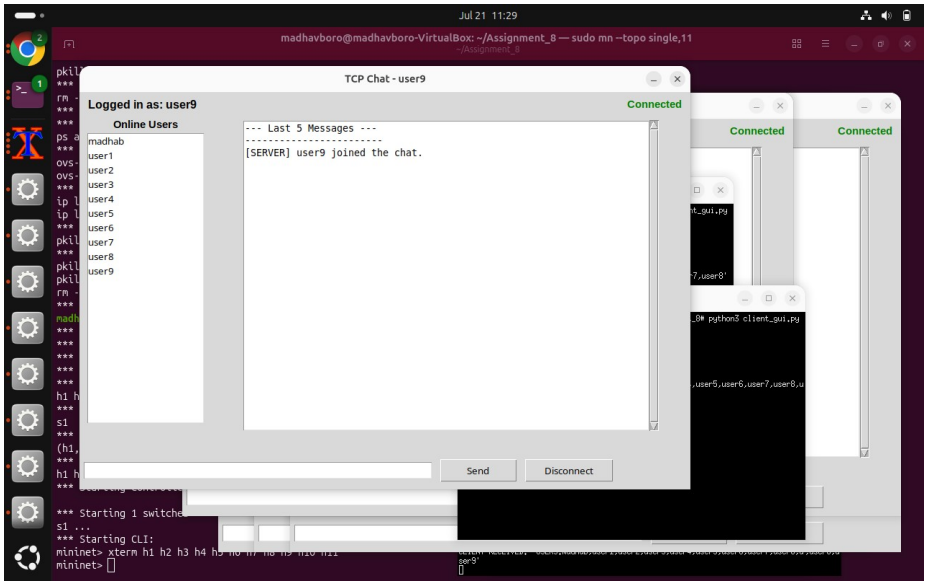

Figure 1: Multiple client windows connected simultaneously in the Mininet topology (h1–h10)

3. Reliability Improvements

The following reliability improvements were implemented:

- Improved exception handling around socket and I/O operations.
- Automatic removal of disconnected clients from the server's active session list.
- Session timeout management, with automatic logout after a period of inactivity.
- Maximum message length validation to reject oversized messages.
- Security/event logging for login attempts and account lockouts.
- Password authentication using SHA-256 hashing.
- Input validation on the username field, and account lockout after repeated failed login attempts.

3.1 Input Validation

Empty usernames are rejected before a login attempt is made:

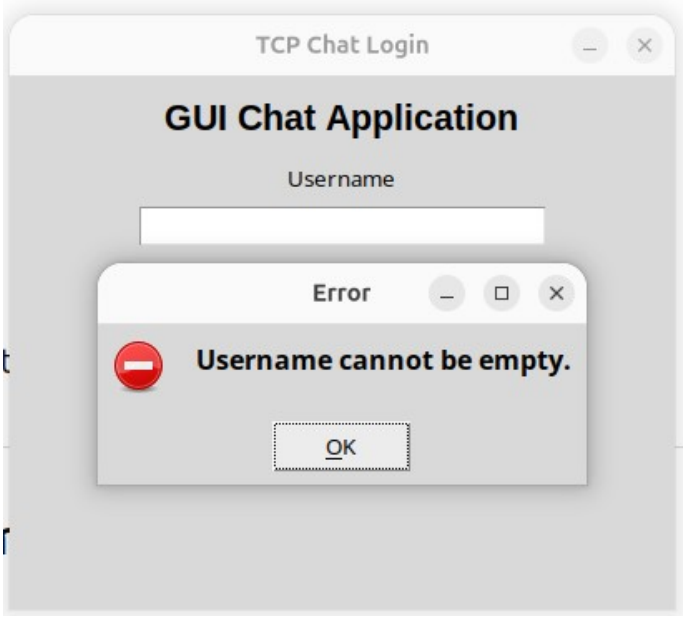

Figure 2: Validation error shown when the username field is left empty

Usernames containing disallowed characters are also rejected client-side:

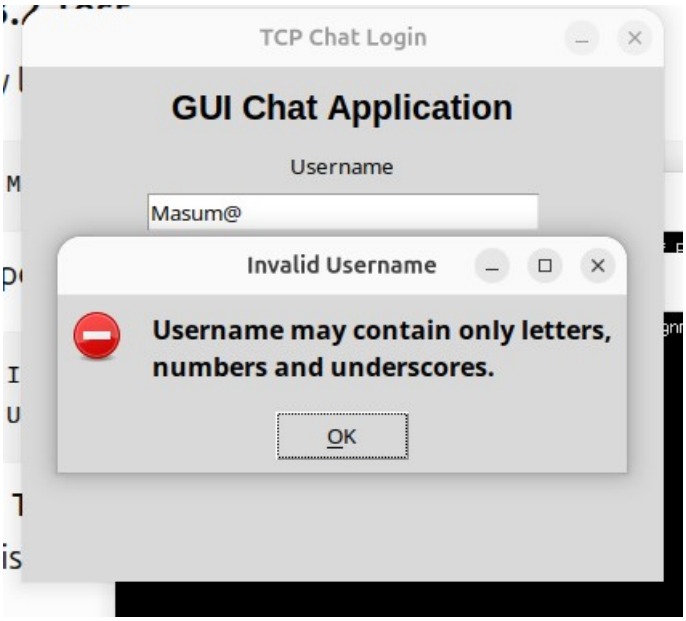

Figure 3: Validation error for a username containing invalid characters

3.2 Authentication and Account Lockout

Invalid username/password combinations are rejected by the server with a clear error message:

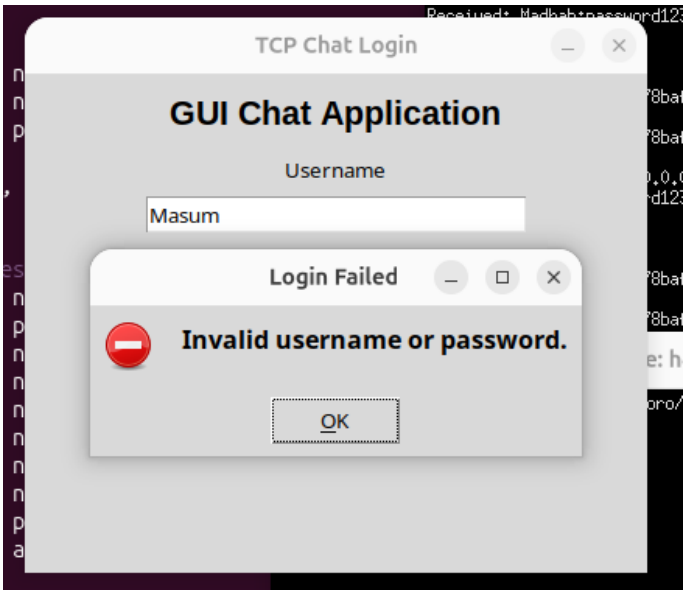

Figure 4: Login failure message for an invalid username or password

After repeated failed login attempts, the account is temporarily locked out to mitigate brute-force login attempts:

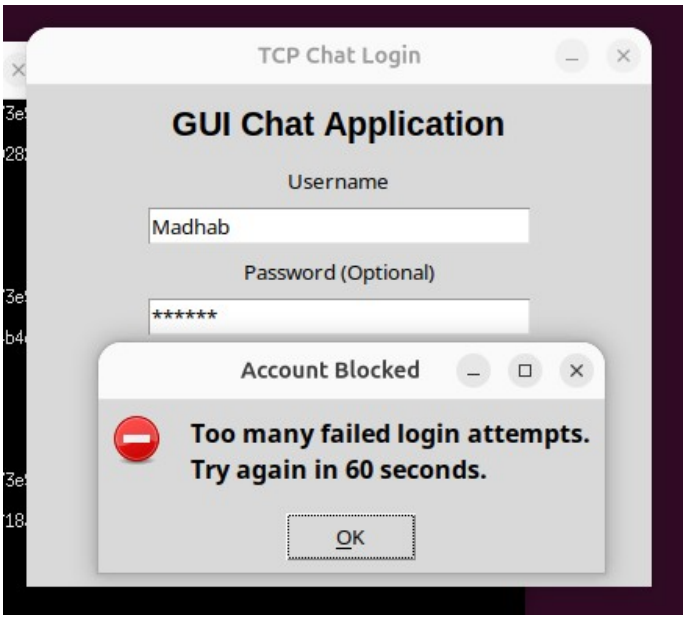

Figure 5: Account temporarily blocked after too many failed login attempts

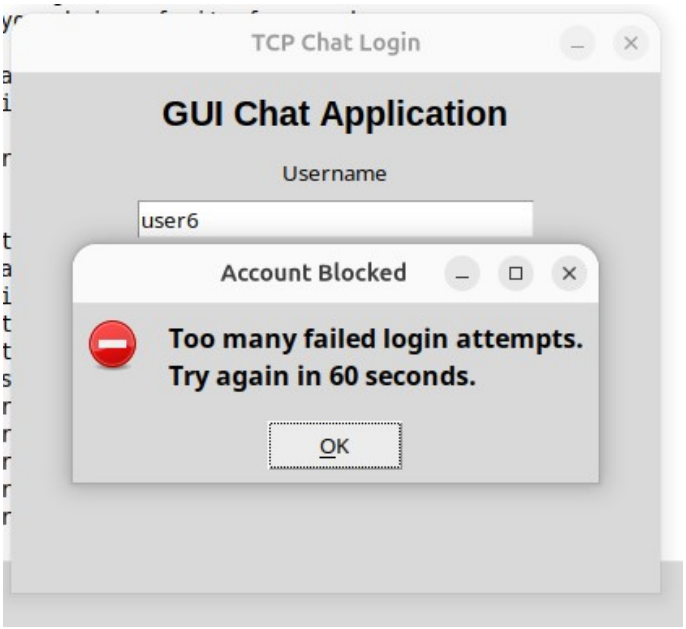

Figure 6: Account lockout verified again during reliability testing

3.3 Session Timeout

Clients that remain inactive for a configured period are automatically logged out, freeing up server-side resources:

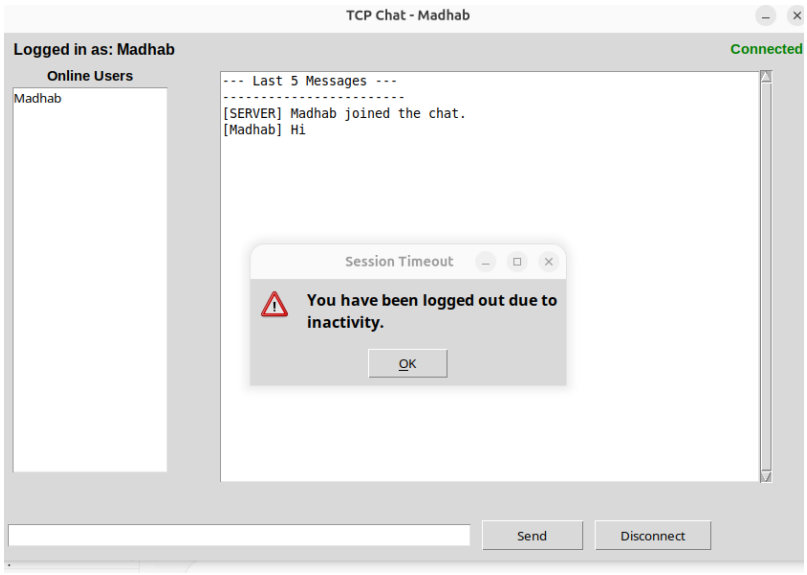

Figure 7: Automatic logout after a period of user inactivity

3.4 Message Length Validation

Messages exceeding the configured maximum length are rejected before being sent to the server:

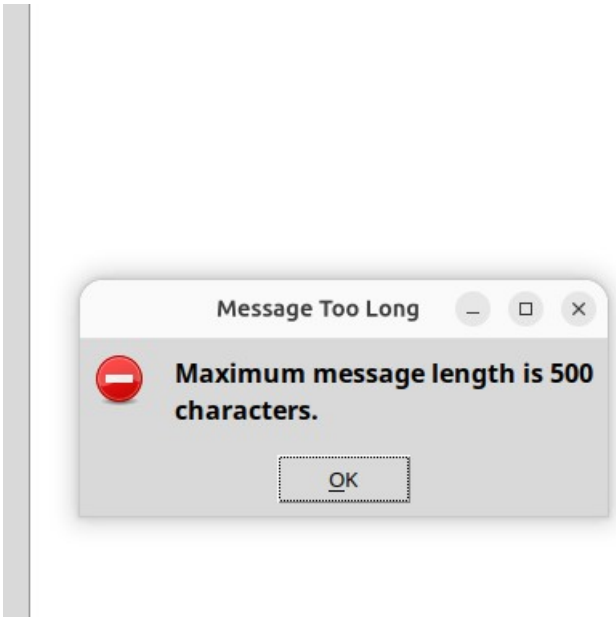

Figure 8: Error shown when a message exceeds the maximum allowed length (500 characters)

4. Configuration Management

All previously hardcoded parameters — including host address, port number, session timeout duration, maximum message length, and login-attempt limits — were moved into an external config.json file. This allows the server and client behaviour to be tuned without modifying or redeploying the application source code, and keeps configuration consistent across all client instances.

5. Experimental Setup

5.1 Software Requirements

- Ubuntu Linux
- Python 3
- Mininet
- Wireshark
- Tkinter
- Pandas
- Matplotlib

5.2 Network Topology

The application was tested using Mininet with the following topology:

```
sudo mn --topo single,11
```

Configuration:

- h1 – Chat Server
- h2 to h11 – Chat Clients

5.3 Existing Implementation Reused

The application from Assignment 7 was reused without redesigning the communication protocol. Existing features include:

- User authentication
- Broadcast messaging
- Private messaging
- Online user list
- Chat history
- Session management

6. Performance Results

6.1 Assignment 7 (Before Optimization)

Online Clients	Messages	Average Delay (ms)	Throughput (msg/sec)
5	10	4131.20	0.24

6.2 Assignment 8 (After Optimization)

Online Clients	Messages	Average Delay (ms)	Throughput (msg/sec)
5	10	12344.66	0.08

Online Clients	Messages	Average Delay (ms)	Throughput (msg/sec)
8	10	108215.01	0.01
10	10	25790.35	0.04

The optimized application was evaluated under different client loads. Although the measured delay increased during these experiments, the primary objective of Assignment 8 was to improve scalability, maintainability and reliability rather than reducing communication latency. The application successfully supported additional concurrent clients while maintaining stable operation.

6.3 Performance Comparison

Feature	Assignment 7	Assignment 8
Configuration	Hardcoded values	External config.json
Client load tested	5 clients	5, 8 and 10 clients
Performance logging	Basic	Automatic CSV generation
Reliability	Basic	Improved exception handling and session timeout
Scalability	Limited	Enhanced scalability testing
Average delay (5 clients)	4131.20 ms	12344.66 ms
Throughput (5 clients)	0.24 msg/sec	0.08 msg/sec

The comparison shows that Assignment 8 introduces several software quality improvements while preserving the original communication protocol. The application becomes easier to configure through config.json, provides improved reliability through exception handling and session timeout, and supports scalability testing with a larger number of concurrent clients. The measured performance values vary due to increased workload and runtime conditions, but the application remains stable under higher client loads.

7. Graph Analysis

7.1 Average Delay vs Online Clients

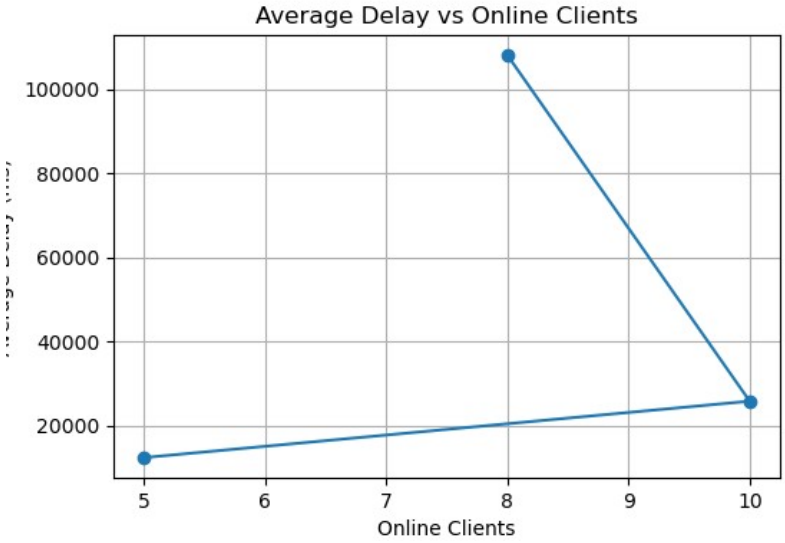

Figure 9: Average delay (ms) as the number of online clients increases

Average delay rose sharply at 8 concurrent clients before dropping again at 10. This non-monotonic pattern suggests the delay figures were influenced by runtime conditions on the test host (e.g., background load, scheduling variance in Mininet) rather than a purely linear scaling effect, and would benefit from averaging over repeated runs.

7.2 Throughput vs Online Clients

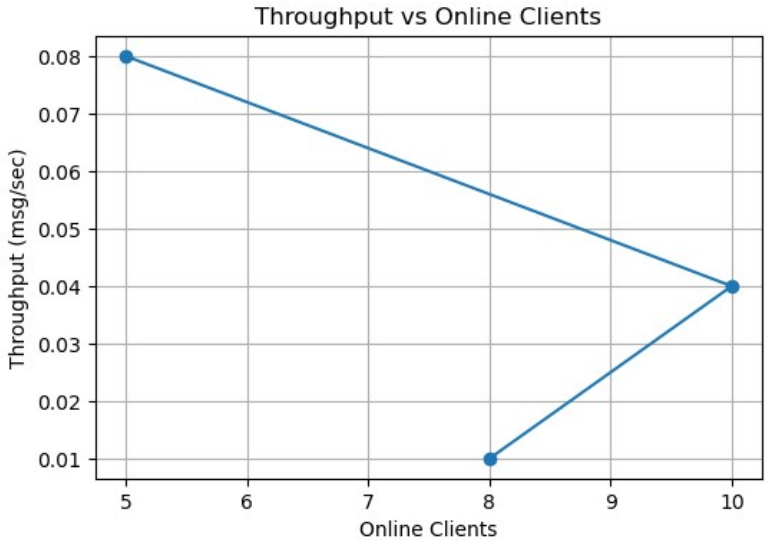

Figure 10: Throughput (msg/sec) as the number of online clients increases

Throughput correspondingly dipped at 8 clients and partially recovered at 10, mirroring the delay trend. While absolute throughput decreased compared to the Assignment 7 baseline, the key result is that the

application remained stable and did not crash at any tested load, which was the primary goal of this assignment.

8. Wireshark Verification

Network traffic was captured using Wireshark to analyze communication between the server and clients. The following events were observed and verified in the capture:

- TCP three-way handshake
- Login authentication
- Broadcast messaging
- Private messaging
- Client logout

The capture confirmed that all client-server communication continued to use the original TCP-based protocol from Assignment 7, with no changes to the message format or connection sequence introduced by the reliability and scalability improvements.

9. Challenges Faced

- Managing multiple concurrent client connections.
- Collecting accurate performance statistics.
- Configuring Mininet with multiple hosts.
- Capturing relevant packets using Wireshark.
- Testing scalability without modifying the communication protocol.

10. Conclusion

The GUI-based chat application from Assignment 7 was successfully enhanced to improve scalability and reliability while preserving the existing TCP communication protocol. Configuration management, exception handling, session timeout, and performance evaluation were added without changing the core application logic. The application was successfully tested with 5, 8 and 10 concurrent clients, demonstrating stable operation under increased load.